

TRABAJO PRÁCTICO FINAL

Introducción a la Inteligencia Artificial

Clasificación Automatizada de Tickets de Soporte Corporativo
Mediante In-Context Learning Utilizando LLaMA

Gastón Dávalos Mateo Piro

July 4, 2026

1 Introducción

El dataset que utilizaremos es Customer Support Tickets, publicado por Tobias Bueck (Tobi-Bueck) en Hugging Face. Se trata de un conjunto de tickets de soporte técnico y atención al cliente etiquetados para tareas de clasificación y análisis de tickets.

El dataset contiene información textual correspondiente a correos electrónicos enviados a servicios de soporte, junto con etiquetas asignadas por agentes de soporte. <https://huggingface.co/datasets/Tobi-Bueck/customer-support-tickets>

El objetivo de nuestro trabajo es hacer que LLaMA prediga el área responsable de distintos conjuntos de texto del dataset de tickets de atención al cliente mediante **In Context Learning**.

Para esto nos basamos en la publicación "Language Models are Few-Shot Learners" (Brown et al., 2020), la cual propone que los grandes modelos pueden resolver tareas específicas extrayendo el conocimiento directamente desde el contexto provisto en el prompt, sin necesidad de modificar sus pesos ni realizar un reentrenamiento.

Vamos a evaluar la eficiencia y alcance de LLaMA para distintos formatos de esta tarea, para esto diseñamos una serie de experimentos variando distintos parámetros. En particular probaremos la eficiencia de LLaMa ante distintas cantidades y variedades de ejemplos, diferentes formatos del prompt y la resistencia al ruido del modelo.

2 Análisis del Dataset

Como ya establecimos previamente el dataset que utilizaremos es Customer Support Tickets, publicado por Tobias Bueck (Tobi-Bueck) en Hugging Face. El dataset contiene tickets en alemán e inglés, aunque vamos a concentrarnos en inglés ya que es el lenguaje con el que principalmente fue entrenada LLaMa.

Cada celda del dataset esta compuesto por:

- Subject: El asunto del mail.
- Body: El texto que escribió el cliente explicando el problema, queja o consulta. Es lo que vamos a pasar al modelo en el prompt.
- Answer: Respuesta provista por el servicio de ayuda.
- Type: El tipo del ticket este puede ser:
 - Incident : Problema inesperado que requiere atención inmediata
 - Request : Consulta rutinaria o solicitud de servicio.
 - Problem : Problema subyacente que causa múltiples incidentes.

- Change : Cambio o actualización planificada.
- Queue: El departamento al cual el ticket tiene que ser enviado. Durante el informe le llamamos categoría. Hay en total 10 opciones:
 - Technical Support
 - Customer Service
 - Billing and Payments
 - Product Support
 - IT Support
 - Returns and Exchanges
 - Sales and Pre-Sales
 - Human Resources
 - Service Outages and Maintenance
 - General Inquiry
- Priority: Low, Medium, Critical
- Language: Indica el idioma del ticket, puede tener dos valores EN(inglés), DE(alemán)
- Version: No relevante
- Tags: Etiquetas o categorías asignadas al ticket para clasificarlo mejor e identificar problemas o temas comunes. Un ticket puede tener multiples types que le corresponden.

En el colab hicimos un par de graficas para visualizar la distribucion de los tickets por categoria, tipo, prioridades o longitud del texto.

3 Metodología

Todos los experimentos fueron realizados utilizando el modelo LLaMA 2 7B en su versión base (meta-llama/Llama-2-7b-hf), implementado mediante la biblioteca Transformers de Hugging Face. Con el objetivo de reducir los requerimientos de memoria y permitir la ejecución en una única GPU, el modelo fue cargado utilizando cuantización de 4 bits mediante la librería BitsAndBytes.

Para todos los experimentos se utilizó decodificación greedy, osea, en cada paso de generación el modelo seleccionó el token con mayor probabilidad. En consecuencia, la generación fue completamente determinística (do_sample = False). La longitud máxima de cada respuesta fue limitada a máximo 20 tokens, cantidad suficiente para generar la categoría correspondiente.

Los prompts fueron contruidos dinámicamente para cada experimento, variando únicamente la característica bajo estudio (formato, cantidad de ejemplos, calidad de los ejemplos, variedad del contexto o presencia de ruido), mientras que el resto de los parámetros permanecieron constantes. De esta forma, las diferencias observadas en el desempeño son causa de la modificación del prompt y no cambios en la configuración del modelo.

En cada ejecución, el prompt fue tokenizado utilizando el tokenizer oficial de LLaMA 2 y enviado al modelo. La salida generada fue posteriormente procesada para extraer la categoría predicha y calcular las métricas correspondientes.

Dependiendo del experimento, cada configuración fue repetida entre 100 y 200 veces, seleccionando aleatoriamente los ejemplos de contexto cuando correspondía, con el objetivo de obtener resultados representativos y reducir el efecto de la variabilidad introducida por la selección de ejemplos.

4 Experimentos

4.1 Diseño del Prompt

Nos preguntamos ¿Cómo influye el diseño del prompt en el desempeño del modelo para la clasificación de tickets?.

Vamos a comparar las siguientes parametros relacionados con el diseno del prompt

- Prompt estructurado y desestructurado.
- Prompt patrón
- Prompt etiquetado

Las variables a medir son:

- Accuracy.
- Exact Match.
- Porcentaje de respuestas inválidas.
- Longitud media de la respuesta.

Esperamos que a medida que el modelo reciba mayor cantidad de información y mas detallada va a poder clasificar mejor siempre y cuando esta información siga una estructura logica y simple. Sin embargo, creemos que eso tiene un límite, si recibe mayor cantidad de información que un cierto límite creemos que el modelo puede empezar a bajar su rendimiento.

4.2 Cantidad de contexto

¿Cómo influye la cantidad de ejemplos proporcionados al modelo mediante In-Context Learning en la calidad de la clasificación?

En este caso queremos determinar si aumentar la cantidad de ejemplos mejora la capacidad de clasificación y hasta qué punto. Vamos a probar con:

- Zero-Shot.
- One-Shot.
- Few-Shot (3 ejemplos).
- Few-Shot (5 ejemplos).
- Many-Shot (10 ejemplos).
- Many-Shot (50 ejemplos).

En este caso pensamos que a mayor cantidad de ejemplos que reciba el modelo va a funcionar mejor, y va a mejorar aun mas si los ejemplos son particularmente de categorías con algo de similitud como lo pueden ser "Sales and Pre-Sales" y "Billing and Payments".

4.3 Calidad del contexto

¿Cómo afecta la calidad de los ejemplos proporcionados en el contexto al desempeño del modelo? Experimentos:

- Todos los ejemplos correctamente etiquetados (0%).
- Un porcentaje de ejemplos con etiquetas incorrectas(25%, 50%, 75%).
- Todos los ejemplos incorrectos(100%).

Objetivo: Analizar la sensibilidad del modelo frente a información errónea dentro del contexto. Creemos que mientras mayor cantidad de ruido tengan los tickets el modelo va a funcionar peor.

4.4 Variedad del contexto

¿Cómo influye la diversidad de los ejemplos incluidos en el contexto sobre la capacidad de generalización del modelo? Experimentos:

- Todos los ejemplos pertenecen a la misma categoría.
- Ejemplos balanceados entre todas las categorías.

Objetivo: Determinar si un contexto más variado ayuda al modelo a comprender mejor la tarea. En este caso pensamos similar a un caso anterior va a funcionar mejor con ejemplos de categorías que tienen similitudes, ya que tendría un ejemplo para discernir entre 2 categorías similares. A su vez, si le damos ejemplos incorrectos ya sea de cualquiera de las 2 formas el modelo va a bajar su rendimiento.

4.5 Robustez frente al ruido

¿Qué tan robusto es el modelo cuando el texto de entrada contiene información irrelevante o ruido? Experimentos:

- Ticket original.
- Ticket con texto irrelevante.
- Ticket con errores tipográficos.

Objetivo: Evaluar si el modelo mantiene su capacidad de clasificación cuando la entrada se aleja de un texto limpio. Creemos que, en líneas generales, al agregar ruido, el modelo va a bajar su rendimiento.

4.6 Predicción de múltiples atributos

¿Cómo afecta la predicción simultánea de múltiples atributos del ticket al desempeño del modelo? Experimentos:

- Categoría sola.
- Categoría y tipo.
- Categoría, tipo y prioridad.

Objetivo: Determinar si LLaMA puede resolver varias tareas de clasificación en un mismo prompt sin perder rendimiento. En este caso pensamos que va a funcionar mejor con menos categorías a clasificar.

4.7 Consideraciones sobre metricas

En todos los experimentos medimos 4 metricas que son: Accuracy, Exact Match y % respuestas invalidas y longitud media respuesta. Es importante destacar que por la naturaleza de LLaMA de ser un modelo de Lenguaje y su funcion decoder only, en algunos casos puede tener comportamientos indeseados.

En nuestro caso cuando en el prompt le ponemos la tarea de que dado un ticket le asigne una categoria sucede recurrentemente 1 de 3 cosas:

- El modelo asigna una categoría ya sea correcta o incorrecta.
- El modelo asigna una categoría y luego imprime texto parecido a un ticket.
- El modelo imprime dicho "ruido" y dentro del mismo le asigna una categoría.

Teniendo en cuenta este comportamiento nuestras métricas aplicadas son las siguientes:

- **Exact Match:** si el modelo imprime solamente la categoría correspondiente sin ningún texto extra.
- **Accuracy:** contamos como un acierto si realiza un exact match o retorna ruido y en algun lugar del ruido(antes o después) imprime la categoría correcta
- Contamos como respuesta inválida si en la respuesta del modelo no hay ninguna categoría válida en cualquier lugar de la misma

5 Resultados

5.1 Diseño del prompt

Para cado caso con prompt estructurado, desestructurado, patrón y etiquetado hicimos lo siguiente: construimos funciones para armar cada prompt y en cada repetición le damos a LLaMa 5 ejemplos al azar y con cada formato tiene que clasificar 1 mismo ticket, eso lo repetimos 200 veces.

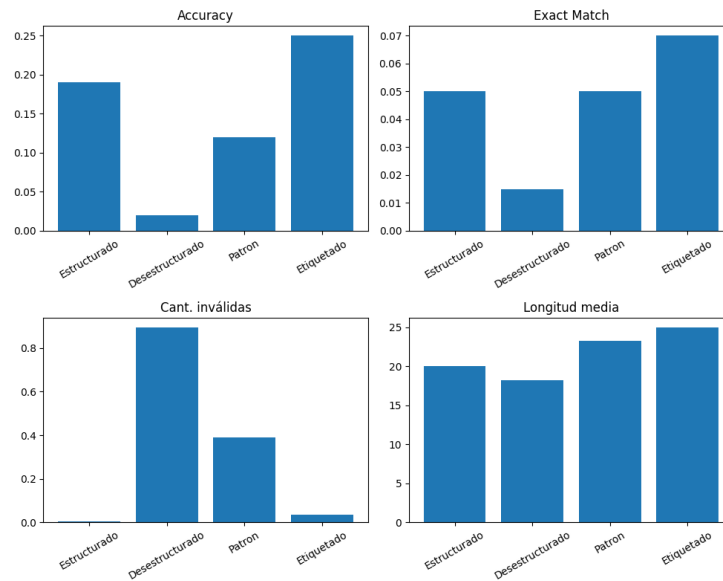


Figure 1: Diseño del prompt

Podemos ver en las gráficas (figura 1) que, en cualquier caso en el que el prompt presentaba algún tipo de segmentacion, como con prompt estructurado, etiquetado o patrón tuvo un rendimiento mucho mayor que en el caso de prompt desestructurado el cual es solo lenguaje natural. Vemos que etiquetado es el que mayor accuracy alcanzó llegando a un aproximadamente 25%, mientras tanto,

tanto el de patrón como el estructurado tienen rendimientos menores, 19% y 12% respectivamente.

Por otro lado el rendimiento de desestructurado es mucho menor a los anteriores siendo 2%, a su vez tiene una gran cantidad de respuestas inválidas siendo estas el 80%, lo sigue patrón con un 4% y luego vienen con mucha menor cantidad estructurado y etiquetada con 0.5% y 3.5% respectivamente.

Finalmente, podemos notar que a medida que la información sigue una estructura clara el modelo produce mejores resultados, siendo las etiquetas el que mejores resultados da. Puede ser que el prompt etiquetado funciona mejor porque tiene símbolos como "<>" que marcan claros delimitadores de las partes del prompt y como el modelo trata de predecir cual es el siguiente token, al pasarle los ejemplos puede aprender que luego de un "<TASK>" deber resolver. Mientras que con lenguaje natural no tiene un patrón tan claro y sigue generando texto y por lo cual genera tantas respuestas invalidas.

5.2 Cantidad de contexto

En este experimento lo que hicimos es cambiar la cantidad de ejemplos que le pasamos al modelo durante el prompt donde por cada cantidad el experimento se repetía 100 veces.

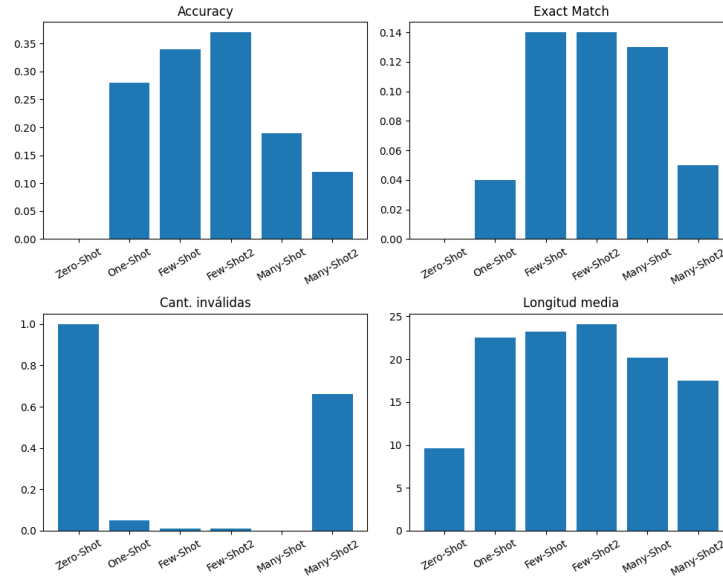


Figure 2: Cantidad de contexto

Se esperaría que a mayor cantidad de ejemplos el modelo tenga una mejora de rendimiento. Sin embargo, podemos observar en las gráficas (figura 2) que se cumple esta hipótesis pero hasta un límite, es decir, el modelo aumenta su

rendimiento a mayor número de ejemplos hasta llegar a 5 ejemplos luego empieza a empeorar.

Por otro lado, vemos que con zero-shot(0 ejemplos) el modelo no tiene ningún acierto lo cual tiene sentido ya que no tiene ningún ejemplo del que pueda aprender retornos válidos.

Una posible explicación de porque a mayor cantidad de ejemplos baja el rendimiento es que un contexto excesivamente largo puede dificultar que el modelo identifique cuáles ejemplos son realmente relevantes para la tarea.

5.3 Calidad del contexto

Debido al experimento anterior en este caso usamos 5 ejemplos los cuales vamos variando cuantos de esos ejemplos son incorrectos eso lo repetimos 100 veces. Uno esperaria que a mayor cantidad de ejemplos incorrectos en el prompt el modelo baje su rendimiento.

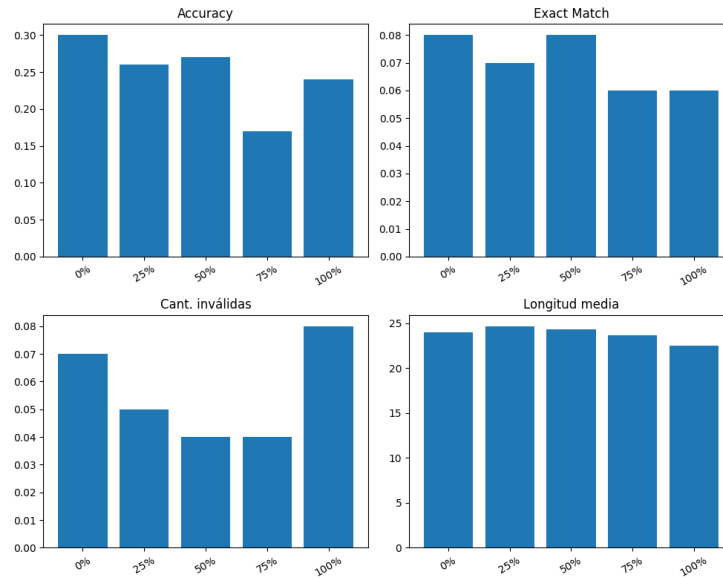


Figure 3: Calidad del contexto

Sin embargo, al ver las gráficas (figura 3) podemos ver que el accuracy no cae fuertemente a medida que aumentamos el numero de ejemplos incorrectos en el prompt. Incluso con la totalidad de ejemplos incorretamente etiquetados (100%) el accuracy no llega a 0% como uno creería que debería pasar.

Esto puede deberse a que el modelo no usa el contenido semántico de los ejemplos, es decir, no usa el body de los ejemplos para aprender que contenido esta relacionado con una categoría en particular, sino en su lugar usa solamente el contenido con el que el modelo fue entrenado.

Por el contrario, usa los ejemplos para aprender el formato de la respuesta que esperamos, es decir, el patrón Queue/Ticket y de esa forma al recibir un Ticket devolver su Queue.

Esto podría explicar también el resultado del experimento anterior, ya que una baja cantidad de ejemplos es suficiente para aprender el formato o patrón y una alta cantidad solo logra confundir al modelo.

5.4 Variedad del contexto

En este caso lo que hicimos fue variar el numero de categorías presentes en los ejemplos del prompt y repetir el experimento 100 veces.

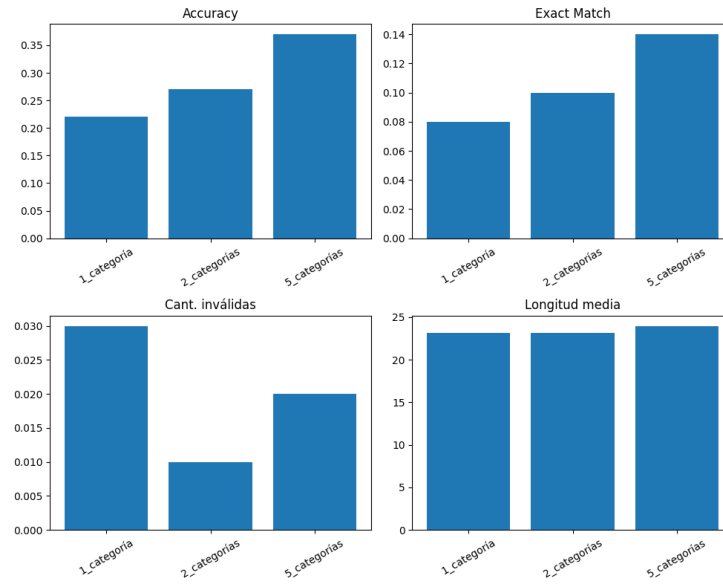


Figure 4: Variedad del Contexto

Vemos claramente que en las gráficas (figura 4) que el accuracy aumenta a medida a medida que tenemos distintas categorías en los ejemplos. A su vez, el exact match se también aumenta lo que sugiere que tener mayor cantidad de categorías presentes en los ejemplos ayuda al modelo a acercarse mas a la respuesta correcta y también ayuda a lograr una coincidencia exacta y limpia con mayor frecuencia. Es interesante notar que aunque la cantidad de respuestas inválidas disminuye a mayor número de categorías hay un aumento entre 2

categorías y 5 categorías, aunque estos valores son muy cercanos al 0%.

En particular vemos que la diferencia entre 1 categoría y 5 categorías es mas de 10% que es una diferencia significativa. Esto puede deberse a que si el modelo recibe solo una categoría puede asociarlo con devolver esta categoría (sin importar el contenido), mientras que con más categorías puede establecer una correspondencia con el contenido del body del ticket y su categoría.

5.5 Robustez frente al ruido

En este experimento lo que tratamos de medir es como se comporta el modelo frente a ruido en el prompt. Para eso lo que hicimos fue entregarle al prompt el ticket con el body normal, con firma (nombre, número telefónico y mail), con palabras añadidas y párrafos añadidos en posiciones aleatorias. Para cada caso lo repetimos al experimento 100 veces.

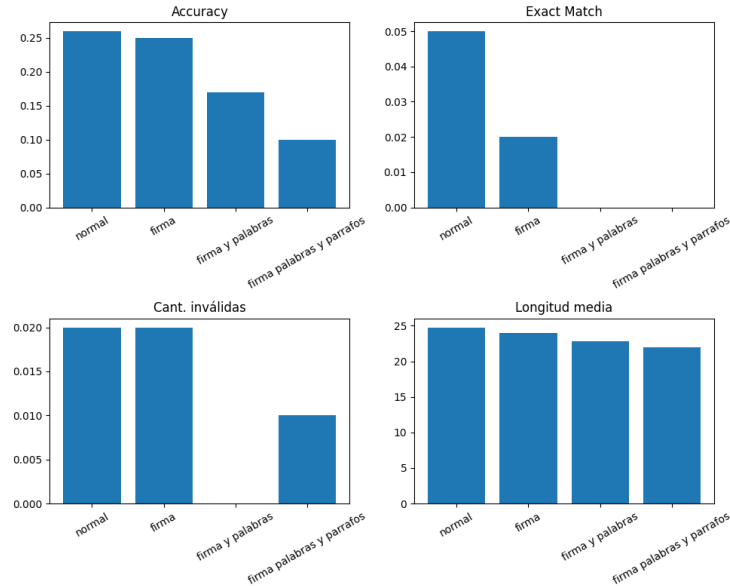


Figure 5: Robustez frente al ruido

Vemos en las gráficas (figura 5) que tanto el accuracy que si bien el accuracy practicamente se mantiene con solamente firma y firma y palabras, el accuracy baja drásticamente cuando a firma y palabras le agregamos párrafos. Lo cual tiene sentido ya que al agregar un párrafo completo en un lugar aleatorio del ticket rompe el sentido semantico del mismo lo que puede llevar al modelo a confundir cual es la información relevante para la clasificación del ticket y cual es simplemente ruido. Por lo tanto, este ruido añade una capa de dificultad

que el modelo tiene que resolver lo que ocasiona que baje su rendimiento. Con lo que, LLaMA soporta ruido como puede ser la firma y palabras sueltas que afectan parcialmente el sentido semántico del ticket mientras que algo como los párrafos que si lo hacen en mayor medida no lo soporta tan bien.

5.6 Predicción de múltiples atributos

En este caso lo que hicimos fue variar el numero de campos válidos y el número de ejemplos presentes y repetir el experimento 100 veces.

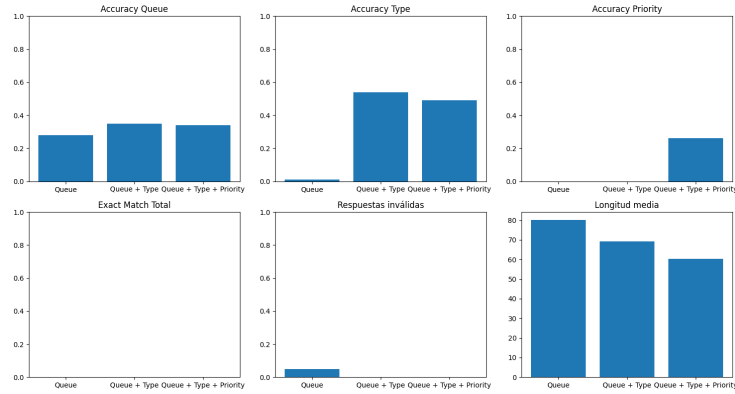


Figure 6: Predicción de múltiples atributos

Vemos en las gráficas (figura 6) que tanto el accuracy en tipo como en prioridad son mucho más fáciles de predecir, ya que a pesar de predecir varios campos simultáneamente estos los predice con mayor accuracy que a la categoría por si sola. Esto nos indica que nuestra creencia de que la categoría es el campo más difícil de predecir es correcta. Creemos que esta dificultad se debe a la cantidad de respuestas posibles (10 categorías, contra 4 tipos o 3 prioridades) y que estas son confusas entre sí, no así como los tipos o prioridades.

6 Conclusiones

Los resultados muestran que el desempeño de LLaMA mediante In-Context Learning depende fuertemente del diseño del prompt. En particular, los prompts estructurados y etiquetados obtuvieron los mejores resultados, mientras que el lenguaje natural produjo una gran cantidad de respuestas inválidas.

Respecto del contexto, observamos que aumentar la cantidad de ejemplos mejora el rendimiento solamente hasta aproximadamente cinco ejemplos. A partir de ese punto el desempeño comienza a disminuir, lo que sugiere que un contexto excesivamente largo puede dificultar la utilización eficiente de la información disponible.

También observamos que la calidad semántica de los ejemplos tuvo un impacto menor al esperado, mientras que la variedad de categorías presentes en el contexto resultó beneficiosa para la clasificación. Esto sugiere que, en esta tarea, el modelo aprovecha principalmente la estructura del prompt y la diversidad de ejemplos para inferir el formato de salida esperado.

Comprobamos también que el modelo es relativamente robusto frente a pequeñas perturbaciones, como firmas o palabras irrelevantes, pero su rendimiento disminuye cuando el ruido altera significativamente la estructura semántica del ticket como con los párrafos.

Finalmente, evaluamos si el modelo puede mantener coherencia cuando debe resolver varias tareas con el mismo contexto, lo cual logra de buena manera.

En conjunto, estos experimentos muestran que el éxito del In-Context Learning depende no solo de la cantidad de información proporcionada, sino también de cómo esta se organiza y presenta al modelo.